

Data Augmentation of Contrastive Learning is Estimating Positive-incentive Noise

Hongyuan Zhang¹ Yanchen Xu^{2,3} Sida Huang^{4,3} Xuelong Li³

Abstract

Inspired by the idea of Positive-incentive Noise (*Pi-Noise* or π -Noise) that aims at learning the reliable noise beneficial to tasks, we scientifically investigate the connection between contrastive learning and π -noise in this paper. By converting the contrastive loss to an auxiliary Gaussian distribution to quantitatively measure the difficulty of the specific contrastive model under the information theory framework, we properly define the task entropy, the core concept of π -noise, of contrastive learning. It is further proved that the predefined data augmentation in the standard contrastive learning paradigm can be regarded as a kind of point estimation of π -noise. Inspired by the theoretical study, a framework that develops a π -noise generator to learn the beneficial noise (instead of estimation) as data augmentations for contrast is proposed. The designed framework can be applied to diverse types of data and is also completely compatible with the existing contrastive models. From the visualization, we surprisingly find that the proposed method successfully learns effective augmentations. Our code is available at <https://github.com/hyzhang98/PiNDA>.

1. Introduction

Contrastive learning (van den Oord et al., 2018; Wu et al., 2018; He et al., 2020; Chen et al., 2020), as a high-performance self-supervised framework, has been intensively investigated in recent years. The contrastive paradigm is even applied to large-scale vision-language models (Rad-

¹The University of Hong Kong ²Fudan University ³Institute of Artificial Intelligence (TeleAI), China Telecom ⁴School of Artificial Intelligence, Optics and ElectroNics (iOPEN), Northwestern Polytechnical University. Correspondence to: Xuelong Li <xuelong_li@ieee.org>.

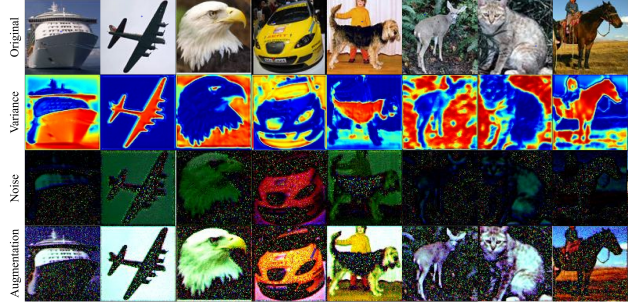


Figure 1. Visualization of the π -noise learned by SimCLR(Chen et al., 2020) with PiNDA on STL-10. We aim to learn the π -noise with a Gaussian distribution $\mathcal{N}(\mu, \Sigma)$. We fix μ as 0 and only learn the variance of Gaussian π -noise, which is visualized in the second row. The third row is a sampling noise and the fourth row contains the images added with π -noise. Compared with other visualizations, the noise is first normalized and then fed into the contrastive module. From the visualization, we find that the π -noise generator successfully learns effective visual augmentations (like style transfer) with only original images.

ford et al., 2021). One of the main limitations of the mainstream contrastive learning models is the dependency of data augmentation. In standard contrastive learning on vision data, the goal of data augmentation is to generate the positive pairs, which is the most crucial element of contrastive learning (Grill et al., 2020). The reliability of data augmentation decides the performance to a great extent (Chen et al., 2020). Inspired by human vision, there are plenty of high-quality augmentation schemes in computer vision, such as rotations, blurring, resizing, cropping, flipping, etc. Therefore, the performance of visual contrastive learning is the most remarkable. SimCLR (Chen et al., 2020), as a typical example, employs various strongly effective visual augmentations to construct positive pairs. CLIP (Radford et al., 2021), as a special example, seems to rely on no visual augmentation. However, as a vision-language model, the language view can be regarded as a cross-modal augmentation of the vision view.

On other types of data, how to generate stable augmentation views becomes one of the hottest research topics. Similar to CLIP, it is easy to extend the contrastive learning into multi-modal and multi-view data (Jiao et al., 2025; Pan & Kang, 2021). Take the graph data as an example: A pri-

mary limitation of graph contrastive learning (Hassani & Khasahmadi, 2020; Zhu et al., 2021) is the instability of random changing the graph structure as augmentations (e.g., dropping edges and nodes, random walk). In the more general scenes where data can be only represented as vectors, some researchers (Cheung & Yeung, 2021; Verma et al., 2021; Yang et al., 2022; Hu et al., 2026) propose to directly utilize noise to generate the augmentations, which is also pointed out by SimCLR (Chen et al., 2020). For example, SimCL (Yang et al., 2022) simply uses random noise to generate the augmentation (Yang et al., 2022) for contrast while the performance is unstable and not satisfactory enough. DACL (Verma et al., 2021) proposes to apply the mixup noise with random ratio to generate positive pairs while MODALS (Cheung & Yeung, 2021) employs an automatic policy search algorithm to learn the mixup ratio for diverse augmentations. Overall, most existing studies of contrastive learning on non-vision data *only view the settings of noise as hyper-parameters* and fail to *learn what kind of noise will be beneficial*, which greatly limits the further promotion of contrastive learning. CLAE (Ho & Vasconcelos, 2020), using adversarial examples as positive pairs, can be regarded as an attempt to learn the profitable perturbation via a heuristic method since it simply uses the perturbation that maximizes the loss.

Inspired by the core idea of Positive-incentive Noise (*Pi-Noise* or π -noise) (Li, 2022; Zhang et al., 2023), we find that the framework of π -noise may offer a reliable scheme to learn stable data augmentations. Specifically speaking, the goal of π -noise is to find the beneficial noise by maximizing the mutual information between the task and noise, i.e.,

$$\max_{\mathcal{E}} \text{MI}(\mathcal{T}, \mathcal{E}) = H(\mathcal{T}) - H(\mathcal{T}|\mathcal{E}), \quad (1)$$

where $\mathcal{T}$ denotes the current task, $\mathcal{E}$ is the noise set, and $\text{MI}(\cdot, \cdot)$ is the mutual information of them. Note that $\mathcal{T}$ is an abstract notation of some random variable since the random variables differs a lot on diverse tasks. Formally, $\mathcal{E}$ subject to $\text{MI}(\mathcal{T}, \mathcal{E}) > 0$ is named as π -noise, while the noise satisfying $\text{MI}(\mathcal{T}, \mathcal{E}) = 0$ is pure noise (Li, 2022). In other words, compared with the random noise, π -noise will not disturb the task.

Motivated by the above intuition, we attempt to investigate **how to scientifically generate stable noisy augmentation views for general contrastive learning** in this paper. The contributions of this paper can be summarized as follows:

- At first, the task entropy of contrastive learning $H(\mathcal{T})$, the core of π -noise framework (Zhang et al., 2023), is defined by designing an auxiliary Gaussian distribution related to contrastive loss. The auxiliary distribution bridges the contrastive loss and information theory.
- With the proper definitions of $H(\mathcal{T})$, we prove that

the predefined data augmentations in the standard contrastive learning framework can be viewed as a point estimation of π -noise, where the prior augmentation is the point-estimated π -noise. From the theoretical analysis of the existing framework, we easily conclude that the point-estimated “ π -noise” is easily designed but it is hard to extend to non-vision data, especially the data that can be only represented by vector.

- Motivated by the connection between π -noise and contrastive learning, we propose a π -noise generator to automatically learn the π -noise as the augmentation, instead of simply point-estimating the “ π -noise” as the existing works, during training the representative module. Since the framework generates π -noise without depending on the specific form of data, the learned π -noise can be used as the augmentation of any type of data, which is not confined to vision data anymore.

Figure 1 shows the π -noise with Gaussian distributions learned by our proposed model. Note that the noise in Figure 1 is normalized to get a clearer visualization before it is fed to contrastive module. In other experiments, the noise is not normalized.

2. Background

2.1. Contrastive Learning

With the rise of self-supervised learning, contrastive learning (van den Oord et al., 2018; Wu et al., 2018; He et al., 2020; Chen et al., 2020; Grill et al., 2020) has been greatly studied. CPC (van den Oord et al., 2018) firstly develops the InfoNCE inspired by the NCE (Gutmann & Hyvärinen, 2010) for contrastive learning. Since the InfoNCE is applied to contrastive learning, plenty of works, such as Memory-Bank (Wu et al., 2018), Moco (He et al., 2020), and SimCLR (Chen et al., 2020), are proposed to greatly improve the performance. In particular, SimCLR proposes and verifies multiple crucial settings for contrastive models, e.g., projection head, embedding normalization for InfoNCE. Some researchers (Robinson et al., 2021; Grill et al., 2020; Cao et al., 2022) focus on how to train the contrastive model more efficiently. For example, (Robinson et al., 2021) points out that more negative samples that are hard to distinguish can accelerate the training, while (Grill et al., 2020; Cao et al., 2022) propose to train the model with only positive samples and they achieve remarkable improvements of performance.

A main limitation of existing contrastive models is the dependency of high-quality reliable data augmentations. For example, SimCLR employs diverse visual augmentations (including rotation, cropping, resizing, blurring, flipping, etc.), and shows that data augmentations are extremely cru-

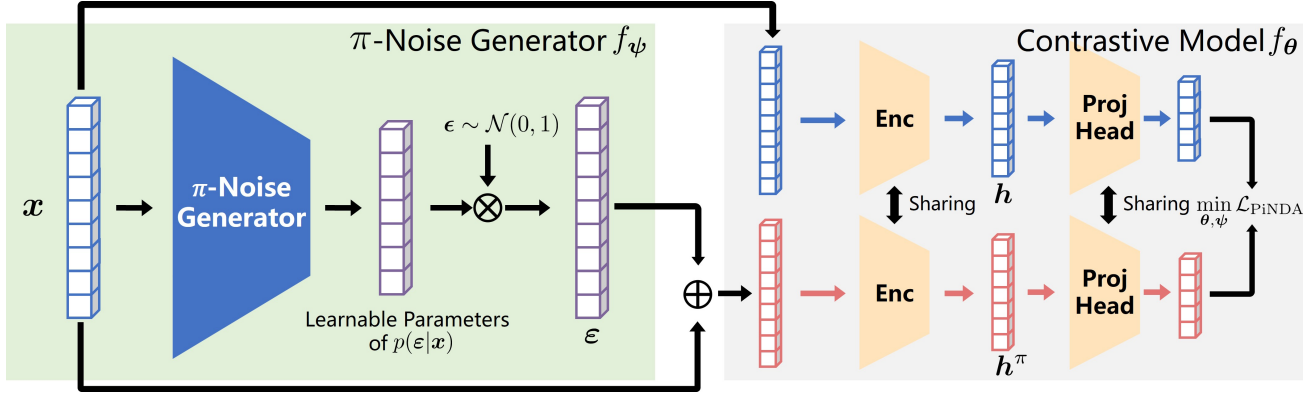


Figure 2. Framework of PiNDA: For simplicity, we assume that there is no other practicable augmentation. So the original sample and π -noise augmentation are used for contrast. If more data augmentations are available, the π -noise can be viewed as one of the augmentations and it will generate and backpropagate gradients to π -noise generator when the π -noise augmentation is sampled.

cial for the performance. When the data can be only represented as vectors, it is harder to obtain augmentations. SimCLR (Chen et al., 2020) finds that the utilization of noise can be a meaningful scheme. Some researchers (Yang et al., 2022; Ho & Vasconcelos, 2020; Verma et al., 2021; Cheung & Yeung, 2021; Wang et al., 2025) formally study how to apply noise to contrastive learning. SimCL (Yang et al., 2022) encodes the original features and then simply add normalized random noise for constructing positive pairs. DACL (Verma et al., 2021) mix up different samples with a random ratio, which is subject to uniform distribution, to generate positive pairs. MODALS (Cheung & Yeung, 2021) designs diverse predefined augmentations (including interpolation, extrapolation, Gaussian noise, and difference transform) and develops an automatic policy search strategy to determine the mixup ratio. CLAE (Ho & Vasconcelos, 2020) proposes to generate adversarial samples instead of random noise as an augmentation view. Overall, most works of how to extend contrastive learning into more general scenes simply view noise as predefined augmentations and fail to scientifically learn the stable and profitable noise in the continuous space.

2.2. Positive-incentive Noise

π -noise (Li, 2022) develops a framework based on information theory to formally claim that the noise may be not always harmful. The principle of maximum the mutual information between task and noise is proposed to learn the π -noise. The key definition in π -noise framework is the task entropy $H(\mathcal{T})$. For example, literature (Zhang et al., 2023; Huang et al., 2025) defines $p(y|x)$, the diversity of labels, as the task distribution to calculate $H(\mathcal{T})$ and then apply the variational inference to the optimization. Compared with it, this paper focuses on a more general and fundamental task, contrastive learning, which requires no supervision. The definition of task entropy in this paper is built on the loss

function, which is more general and scalable.

3. Proposed Method

In this section, we will first introduce the preliminary knowledge. Then the main theoretical derivations and conclusion of the relation between contrastive learning and π -noise is shown. Finally, the π -Noise Driven Data augmentations induced by the theoretical analysis will be elaborated. The whole framework is illustrated in Figure 2.

3.1. Preliminary of Contrastive Learning

Before the formal analysis, we first summarize the popular contrastive learning with InfoNCE (van den Oord et al., 2018) as follows

$$\min_{\theta} -\frac{1}{|\mathcal{D}|} \sum_{\mathbf{x} \in \mathcal{D}} \log \frac{\ell_{\text{pos}}(\mathbf{x}; \theta)}{\ell_{\text{pos}}(\mathbf{x}; \theta) + \ell_{\text{neg}}(\mathbf{x}; \theta)} = \mathcal{L}_{\text{InfoNCE}}, \quad (2)$$

where $\ell_{\text{pos}}(\mathbf{x}; \theta)$ and $\ell_{\text{neg}}(\mathbf{x}; \theta)$ respectively denote the positive and negative loss of sample $\mathbf{x}$ parameterized by θ , $\mathcal{D}$ represents the dataset, and $|\mathcal{D}|$ is the dataset size. There are delicate differences of ℓ_{pos} and ℓ_{neg} among the mainstream contrastive paradigms (van den Oord et al., 2018; Wu et al., 2018; He et al., 2020; Chen et al., 2020; Radford et al., 2021). For instance, let $\mathbf{x}_+$, $\mathbf{x}'_+$ be the positive samples of $\mathbf{x}$, f_{θ} be the learnable non-linear representative module, and τ be the temperature coefficient. The positive loss of SimCLR (Chen et al., 2020) is

$$\ell_{\text{pos}}^{\text{SimCLR}}(\mathbf{x}; \theta) = \exp(\text{sim}(\mathbf{x}_+, \mathbf{x}'_+)/\tau), \quad (3)$$

where $\text{sim}(\mathbf{u}, \mathbf{v}) = f_{\theta}(\mathbf{u})^T f_{\theta}(\mathbf{v}) / (\|\mathbf{u}\| \cdot \|\mathbf{v}\|)$, while the MemoryBank (Wu et al., 2018) uses

$$\ell_{\text{pos}}^{\text{MB}}(\mathbf{x}; \theta) = \exp(f_{\theta}(\mathbf{x})^T f_{\theta}(\mathbf{x}_+)/\tau) \quad (4)$$

as the positive loss. Similarly, the negative loss and choice of negative pairs are also technically different from each

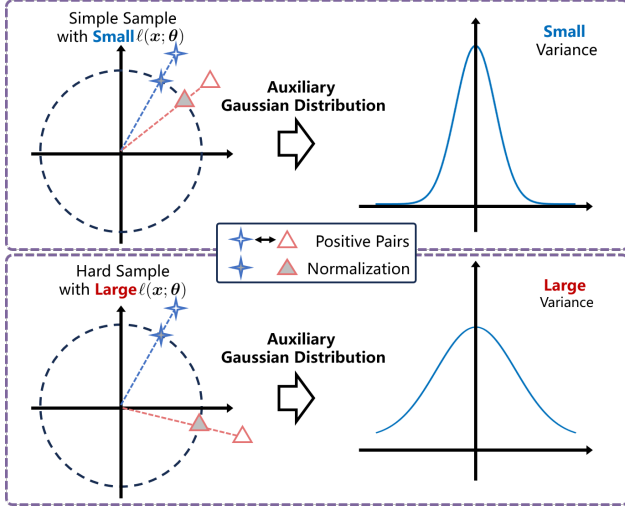


Figure 3. Illustration of the auxiliary Gaussian distribution. The smaller contrastive loss (i.e., larger $\gamma_{\theta^*}(\mathbf{x}, \epsilon)$) is converted to a Gaussian distribution with smaller variance (i.e., smaller entropy) and vice versa.

other.

To state the following analysis, we separate the contrastive loss and define the contrastive loss of each data point $\mathbf{x}$ as

$$\ell(\mathbf{x}; \theta) = -\log \frac{\ell_{\text{pos}}(\mathbf{x}; \theta)}{\ell_{\text{pos}}(\mathbf{x}; \theta) + \ell_{\text{neg}}(\mathbf{x}; \theta)}, \quad (5)$$

so that $\mathcal{L}_{\text{InfoNCE}} = 1/|\mathcal{D}| \cdot \sum_{\mathbf{x}} \ell(\mathbf{x}; \theta)$.

3.2. Formulate Task Entropy via Contrastive Loss

It seems intractable to precisely design the task entropy of contrastive learning with different contrastive models as the original definition. However, it seems much easier to measure the difficulty of contrastive learning on a **specific contrastive model** and a **given dataset**. Clearly, given a dataset, $\sum_{\mathbf{x}} \ell(\mathbf{x}; \theta^*)$ is a straightforward quantity to reflect how difficult the contrastive learning task on the specific dataset is, where θ^* is the optimal parameter. In other words, the smaller $\sum_{\mathbf{x}} \ell(\mathbf{x}; \theta^*)$ means the easier contrastive task and vice versa. However, $\ell(\mathbf{x}; \theta^*)$ is neither a random variable nor a probability quantity so that it can not be directly applied to Eq. (1). To bridge the framework of π -noise and the complexity metric $\ell(\mathbf{x}; \theta^*)$, we introduce **an auxiliary random variable** α subject to $\alpha|\mathbf{x} \sim \mathcal{N}(0, \gamma_{\theta^*}(\mathbf{x})^{-1})$. i.e.,

$$p(\alpha|\mathbf{x}) = \mathcal{N}(0, \gamma_{\theta^*}(\mathbf{x})^{-1}), \quad (6)$$

where

$$\gamma_{\theta^*}(\mathbf{x}) = \frac{\ell_{\text{pos}}(\mathbf{x}; \theta^*)}{\ell_{\text{pos}}(\mathbf{x}; \theta^*) + \ell_{\text{neg}}(\mathbf{x}; \theta^*)} = \exp(-\ell(\mathbf{x}; \theta^*)). \quad (7)$$

The information entropy of the above auxiliary distribution $p(\alpha|\mathbf{x})$ reflects the complexity of learning representation

of $\mathbf{x}$ via contrastive models. Accordingly, the task entropy of contrastive learning on a given distribution $p(\mathbf{x})$ can be formulated as

$$H(\mathcal{T}) = \mathbb{E}_{\mathbf{x} \sim p(\mathbf{x})} H(\mathcal{N}(0, \gamma_{\theta^*}(\mathbf{x})^{-1})). \quad (8)$$

Note that $H(\mathcal{T})$ is lower-bounded by $H(\mathcal{N}(0, 1))$ due to that $\gamma_{\theta^*}(\mathbf{x}) \in [0, 1]$. If $H(\mathcal{T})$ is required to reach the minimum of entropy, the definition of auxiliary variable can be $\kappa(\gamma_{\theta^*}(\mathbf{x}))$, where $\kappa: [0, 1] \mapsto [0, \infty)$ is a monotonically increasing function. However, the range of $H(\mathcal{T})$ does not affect the theoretical result and optimization. We thereby just use the simple formulation shown in Eq. (8).

Note that $\gamma_{\theta^*}(\mathbf{x})$ is also related to the predefined data augmentations, while we omit it to simplify the notations and discussions. In the following subsections, we will show **the role of data augmentations under our framework**.

Remark: We omit the hyper-parameters (e.g., temperature coefficient τ in Eq. (3) and (4)) above. It is easy and direct to extend the definitions to the loss with hyper-parameters. For example, the auxiliary distribution of the loss $\ell(\mathbf{x}; \theta, \tau)$ can be directly defined as $\mathcal{N}(0, \gamma_{\theta^*}(\mathbf{x}; \tau)^{-1})$ where τ^* is the optimal hyper-parameter for the given model and dataset. However, to keep the notations uncluttered, the subscripts for hyper-parameters are omitted in the following part.

3.3. Connection of π -Noise and Contrastive Learning

According to the definition of the task entropy $H(\mathcal{T})$, the mutual information can be formulated as

$$\begin{aligned} \text{MI}(\mathcal{T}, \mathcal{E}) &= \mathbb{E}_{\mathbf{x} \sim p(\mathbf{x})} \int p(\alpha, \epsilon|\mathbf{x}) \log \frac{p(\alpha, \epsilon|\mathbf{x})}{p(\alpha|\mathbf{x})p(\epsilon|\mathbf{x})} d\alpha d\epsilon \\ &= \int p(\alpha, \epsilon, \mathbf{x}) \log \frac{p(\alpha, \epsilon|\mathbf{x})}{p(\alpha|\mathbf{x})p(\epsilon|\mathbf{x})} d\alpha d\epsilon d\mathbf{x}. \end{aligned}$$

As shown in Eq. (1), the conditional entropy $H(\mathcal{T}|\mathcal{E})$ to minimize can be formulated as $H(\mathcal{T}|\mathcal{E}) =$

$$-\int p(\alpha|\mathbf{x}, \epsilon) p(\epsilon|\mathbf{x}) p(\mathbf{x}) \log p(\alpha|\mathbf{x}, \epsilon) d\mathbf{x} d\epsilon d\alpha. \quad (9)$$

Since the dataset $\mathcal{D}$ can be regarded as a sampling from $p(\mathbf{x})$, we can apply the Monte Carlo method to the above formulation and the conditional entropy can be estimated as $-H(\mathcal{T}|\mathcal{E}) \approx$

$$\frac{1}{n} \sum_{\mathbf{x}} \int p(\alpha|\mathbf{x}, \epsilon) p(\epsilon|\mathbf{x}) \log p(\alpha|\mathbf{x}, \epsilon) d\epsilon d\alpha. \quad (10)$$

On the right hand of the above formulation, there are two probabilities: $p(\alpha|\mathbf{x}, \epsilon)$ and $p(\epsilon|\mathbf{x})$. $p(\epsilon|\mathbf{x})$ is the distribution (with some learnable parameters) of π -noise to learn. Therefore, it is a crucial step to model $p(\alpha|\mathbf{x}, \epsilon)$. We first define $\ell_{\text{pos}}(\mathbf{x}, \epsilon; \theta^*)$ and $\ell_{\text{neg}}(\mathbf{x}, \epsilon; \theta^*)$ as the contrastive

loss with augmentation views **generated by** ϵ . Then, we can define the auxiliary distribution with ϵ (shown in Section 3.2) as

$$p(\alpha|\mathbf{x}, \epsilon) = \mathcal{N}(0, \gamma_{\theta^*}(\mathbf{x}, \epsilon)^{-1}), \quad (11)$$

where

$$\gamma_{\theta^*}(\mathbf{x}, \epsilon) = \frac{\ell_{\text{pos}}(\mathbf{x}, \epsilon; \theta^*)}{\ell_{\text{pos}}(\mathbf{x}, \epsilon; \theta^*) + \ell_{\text{neg}}(\mathbf{x}, \epsilon; \theta^*)}.$$

The intuition of the auxiliary distribution is illustrated in Figure 3.

To keep the simplicity of discussion, we assume that only one augmentation is used for contrast. It is not hard to verify that the following analysis still holds with multiple augmentations. If the data augmentation is viewed as a predefined special *noise*, then the augmentation is essentially equivalent to the following assumption

$$p(\epsilon|\mathbf{x}) \rightarrow \delta_{\epsilon_0}(\epsilon), \quad (12)$$

where $\delta_{\epsilon_0}(\epsilon)$ is the Dirac delta function with translation ϵ_0 , i.e., $\delta_{\epsilon_0}(\epsilon)$ is not zero if and only if $\epsilon = \epsilon_0$. It is **the point estimation of** $p(\epsilon|\mathbf{x})$, which is widely used in Maximum A Posteriori (MAP) (Bishop, 2007).

Owing to the point estimation, $-H(\mathcal{T}|\mathcal{E})$ can be further simplified as

$$-H(\mathcal{T}|\mathcal{E}) \approx \frac{1}{n} \sum_{\mathbf{x}} \int p(\alpha|\mathbf{x}, \epsilon_0) \log p(\alpha|\mathbf{x}, \epsilon_0) d\alpha = \mathcal{L}.$$

The probability density function (PDF) of $\mathcal{N}(0, \gamma_{\theta^*}(\mathbf{x}, \epsilon_0)^{-1})$ in $\mathcal{L}$ is formulated as

$$p(\alpha|\mathbf{x}, \epsilon_0) = C \sqrt{\gamma_{\theta^*}(\mathbf{x}, \epsilon_0)} \exp\left(-\frac{\alpha^2}{2} \cdot \gamma_{\theta^*}(\mathbf{x}, \epsilon_0)\right),$$

where C represents some constant term in Gaussian distribution. Substitute the PDF into $\mathcal{L}$, we have

$$\mathcal{L} = \frac{1}{n} \sum_{\mathbf{x}} \log C + \frac{1}{2} \log \gamma_{\theta^*}(\mathbf{x}, \epsilon_0) - \frac{1}{2}. \quad (13)$$

Due to the page limitation, more details can be found in Appendix A. Since the optimal θ^* is not known and it is the learnable parameters of the contrastive model, it is a direct scheme to *simultaneously learn θ during finding the π -noise $\mathcal{E}$* . Specifically, the formal optimization objective to find π -noise for contrastive learning is

$$\max_{\mathcal{E}, \theta} \text{MI}(\mathcal{T}, \mathcal{E}). \quad (14)$$

Through the above derivations and estimations, the original goal to maximize the mutual information is converted to

$$\max_{\mathcal{E}, \theta} \text{MI}(\mathcal{T}, \mathcal{E}) \Rightarrow \max_{\theta} \mathcal{L} \Leftrightarrow \max_{\theta} \frac{1}{n} \sum_{\mathbf{x}} \log \gamma_{\theta}(\mathbf{x}, \epsilon_0)$$



Figure 4. Visualization of different noise settings on SimCLR: In the second and third rows, we visualize the learnable μ and Σ of Gaussian noise respectively. The fourth row shows the learned π -noise from the uniform distribution. The last row is the learned Gaussian noise with $\mu = 0$.

$$\Leftrightarrow \min \frac{1}{n} \sum_{\mathbf{x}} -\log \frac{\ell_{\text{pos}}(\mathbf{x}, \epsilon_0; \theta)}{\ell_{\text{pos}}(\mathbf{x}, \epsilon_0; \theta) + \ell_{\text{neg}}(\mathbf{x}, \epsilon_0; \theta)}, \quad (15)$$

which is the same as the standard contrastive learning paradigm. In conclusion, the standard contrastive learning paradigm is equivalent to **optimize the contrastive learning module with point-estimated π -noise** where *the predefined data augmentation is the point estimation of π -noise*.

3.4. π -Noise Driven Data Augmentation

From the derivations presented in the previous subsection, there is a natural scheme to enhance the existing contrastive learning framework: automatically learn the π -noise, i.e., Pi-Noise Driven Data Augmentation (PiNDA).

The advantages of learning π -noise as augmentations mainly includes: (1) Plenty of existing contrastive models (e.g., SimCLR) also use the images with untrained noise as one of the predefined data augmentations and π -noise driven data augmentations offer a more stable scheme; (2) As the training of π -noise does not depend on the structure of data, it offers a reliable augmentation scheme for non-vision data. It should be emphasized that PiNDA is completely compatible with the existing contrastive models.

Formally, instead of applying the point-estimation of π -noise as shown in Eq. (12), we propose to directly learn $p_{\psi}(\epsilon|\mathbf{x})$ parameterized by ψ . The negative conditional entropy estimated by the Monte Carlo method (given by Eq. (10)) can be rewritten as $-H(\mathcal{T}|\mathcal{E}) \approx$

$$\frac{1}{n} \sum_{\mathbf{x}} \int \mathbb{E}_{\epsilon \sim p_{\psi}(\epsilon|\mathbf{x})} p(\alpha|\mathbf{x}, \epsilon) \log p(\alpha|\mathbf{x}, \epsilon) d\alpha. \quad (16)$$

The above expectation formulation can be also estimated by the Monte Carlo method. Furthermore, to backpropagate the gradient to $p_{\psi}(\epsilon|\mathbf{x})$, the reparameterization trick (Kingma & Welling, 2014) is applied. In other words, $p(\epsilon)d\epsilon = p(\epsilon|\mathbf{x})d\epsilon$ where $\epsilon = f_{\psi}(\mathbf{x}, \epsilon)$, f_{ψ} is the π -noise generator,

Algorithm 1 Pseudo code of compute the contrastive loss with PiNDA

Input: A sample $\mathbf{x}$, batch size m , an augmentation $a(\cdot)$.
 Sample an auxiliary random noise $\epsilon \sim p(\epsilon)$.
 Generate π -noise: $\epsilon = f_\psi(\mathbf{x}, \epsilon)$.
 # π -noise augmentation
 Get a noisy augmentation $\mathbf{h}^\pi = f_\theta(\mathbf{x} + \epsilon)$
 $\mathbf{h}' = f_\theta(a(\mathbf{x}))$.
 Compute $\mathcal{L}_{\text{PiNDA}}$ using $\mathbf{h}^\pi$ and $\mathbf{h}'$ as the condition $(\mathbf{x}, \epsilon)$ in Eq. (17).
 $\ell_{\text{PiNDA}}(\mathbf{x}; \theta, \psi) = -\log \frac{\ell_{\text{pos}}(\mathbf{x}, \epsilon; \theta)}{\ell_{\text{pos}}(\mathbf{x}, \epsilon; \theta) + \ell_{\text{neg}}(\mathbf{x}, \epsilon; \theta)}$
Output: $\ell_{\text{PiNDA}}(\mathbf{x}; \theta, \psi)$.

and $p(\epsilon)$ is a standard Gaussian distribution. Accordingly, the loss of PiNDA is formulated as

$$\begin{aligned} \mathcal{L}_{\text{PiNDA}} &= -\frac{1}{n} \sum_{\mathbf{x}} \mathbb{E}_{\epsilon \sim p(\epsilon)} \int p(\alpha | \mathbf{x}, \epsilon) \log p(\alpha | \mathbf{x}, \epsilon) d\alpha \\ &= -\frac{1}{n} \sum_{\mathbf{x}} \mathbb{E}_{\epsilon \sim p(\epsilon)} \log \frac{\ell_{\text{pos}}(\mathbf{x}, \epsilon; \theta)}{\ell_{\text{pos}}(\mathbf{x}, \epsilon; \theta) + \ell_{\text{neg}}(\mathbf{x}, \epsilon; \theta)}. \end{aligned} \quad (17)$$

It should be pointed out that θ and ψ represent the parameters of the contrastive model and π -noise generator, respectively. We summarize the whole procedure of a base contrastive model with PiNDA as the only augmentation in Algorithm 1. The pseudo code of a typical contrastive model with PiNDA and other predefined augmentations is summarized in Algorithm 2.

4. Experiments

In this section, we perform experiments to investigate *whether the proposed π -noise augmentation could promote the performance of the existing contrastive learning models*. As clarified in the previous sections, a primary merit of PiNDA is the generalization. In other words, PiNDA is not limited to vision data.

Table 1. Details of 4 non-vision datasets and 5 vision datasets.

Dataset	Vision	Size	Classes
HAR	✗	10,299	6
Reuters	✗	10,000	4
Reuters-21578	✗	11,228	46
Epsilon	✗	500,299	2
MSLR-WEB30K	✗	10,558,186	5
CIFAR-10	✓	60,000	10
CIFAR-100	✓	60,000	100
Fashion-MNIST	✓	70,000	10
STL-10	✓	113,000	10
ImageNet	✓	1,331,167	1000

Algorithm 2 Pseudo code of PiNDA with other augmentations

Input: Dataset $\mathcal{D}$, batch size m , predefined augmentations $\mathcal{A}$.
if $\mathcal{A} = \emptyset$ **then**
 # If no predefined augmentation is available, use the original view as another augmentation.
 $\mathcal{A} = \mathcal{A} \cup \{\text{Identity function}\}$.
end if
 $\mathcal{A} = \mathcal{A} \cup \{\text{PiNDA}\}$.
for A sampled batch $\{\mathbf{x}_i\}_{i=1}^m$ **do**
for each $\mathbf{x}_i$ **do**
 Draw two augmentations $a \sim \mathcal{A}$ and $a' \sim \mathcal{A}$.
if PiNDA $\in \{a, a'\}$ **then**
 Compute $\ell_{\text{PiNDA}}(\mathbf{x}_i; \theta, \psi)$ by Algorithm 1.
 $\mathcal{L} = \mathcal{L} + \ell_{\text{PiNDA}}(\mathbf{x}_i; \theta, \psi)$.
else
 Compute $\ell(\mathbf{x}_i; \theta)$ and $\mathcal{L} = \mathcal{L} + \ell(\mathbf{x}_i; \theta)$.
end if
end for
 Update θ and ψ to minimize $\mathcal{L}$.
end for
Output: Contrastive model f_θ and π -noise generator f_ψ .

4.1. Datasets

In our experiments, there are 5 non-vision datasets used for validating the effectiveness of PiNDA on non-vision data: HAR (Reyes-Ortiz et al., 2012), Reuters and Reuters-21578 (Lewis et al., 2004), Epsilon (PASCAL, 2008), and MSLR-WEB30K (Qin & Liu, 2013). In addition, 5 vision datasets, including Fashion-MNIST (Xiao et al., 2017), CIFAR-10 (Torralba et al., 2008), CIFAR-100 (Torralba et al., 2008), STL-10 (Coates et al., 2011), and ImageNet (Russakovsky et al., 2015), are also used for evaluation. On the one hand, *the experiments on these vision datasets provide the better visualization of π -noise augmentation*. On the other hand, *the experiments also validates the effectiveness and compatibility of PiNDA to the existing contrastive models*. The details are summarized in Table 1. More details can be found in Appendix B.1.

4.2. Experimental Settings

Baseline For **non-vision datasets**, SimCL (Yang et al., 2022) and CLAE (Ho & Vasconcelos, 2020) serve as the competitors. *SimCL* adds the normalized noise to the learned representations to obtain augmentations, while *CLAE* (Ho & Vasconcelos, 2020) uses adversarial examples as an augmentation. To better *verify whether the idea of learning π -noise works*, a compared method that utilizes the untrained random Gaussian noise is also added as a competitor method, denoted by *Random Noise*. *SimCLR* (Chen et al., 2020),

BYOL (Grill et al., 2020), *SimSiam* (Chen & He, 2021), *MoCo* (He et al., 2020), and *DINO* (Caron et al., 2021) also serve as important baselines on **vision data**.

Backbone On non-vision datasets, the contrastive backbone is set to a 3-layer fully connected neural network since no efficient model can be applied. The dimension of each hidden layer is set to 1024 and the dimension of the final embedding is 256. On vision datasets, we use ResNet18 (He et al., 2016) as the backbone for contrastive representation learning for all the models by default. Moreover, we also use ResNet50 (He et al., 2016) as the backbone for some models to test the generalization of PiNDA.

Table 2. Test accuracy (%) averaged over 5 runs on 4 non-vision datasets evaluated by k NN and Softmax Regression (SR). All superscripts and subscripts represent the specific settings. The best results are highlighted in bold.

Dataset	Method	k NN	SR
HAR	Random Noise	77.76 $\pm$ 0.61	77.62 $\pm$ 0.27
	SimCL	61.12 $\pm$ 3.21	63.92 $\pm$ 0.16
	PiNDA $_{\mu=0}$	77.14 $\pm$ 0.94	86.20$\pm$0.40
	PiNDA $_{\mu \neq 0}$	78.09$\pm$0.75	81.33 $\pm$ 0.60
	CLAE	85.71 $\pm$ 0.18	90.80 $\pm$ 0.66
	PiNDA $_{\text{CLAE}}^{\mu=0}$	86.34$\pm$0.70	91.10$\pm$0.35
	PiNDA $_{\text{CLAE}}^{\mu \neq 0}$	85.89 $\pm$ 0.01	90.52 $\pm$ 0.59
Reuters	Random Noise	82.84 $\pm$ 6.02	77.30 $\pm$ 7.48
	SimCL	64.20 $\pm$ 4.19	73.63 $\pm$ 0.05
	PiNDA $_{\mu=0}$	84.43 $\pm$ 1.23	86.03$\pm$0.48
	PiNDA $_{\mu \neq 0}$	86.37$\pm$1.07	82.50 $\pm$ 0.22
	CLAE	82.60 $\pm$ 0.33	78.03 $\pm$ 0.36
	PiNDA $_{\text{CLAE}}^{\mu=0}$	82.37 $\pm$ 1.07	79.03 $\pm$ 0.66
	PiNDA $_{\text{CLAE}}^{\mu \neq 0}$	82.80$\pm$0.01	79.07$\pm$0.66
Epsilon	Random Noise	52.00 $\pm$ 0.30	60.21 $\pm$ 0.28
	SimCL	50.90 $\pm$ 0.36	59.49 $\pm$ 0.01
	PiNDA $_{\mu=0}$	53.20$\pm$0.08	61.53$\pm$0.08
	PiNDA $_{\mu \neq 0}$	52.64 $\pm$ 0.34	60.21 $\pm$ 0.28
	CLAE	51.91 $\pm$ 0.35	59.17 $\pm$ 0.61
	PiNDA $_{\text{CLAE}}^{\mu=0}$	52.31$\pm$0.29	59.38 $\pm$ 0.32
	PiNDA $_{\text{CLAE}}^{\mu \neq 0}$	51.70 $\pm$ 0.18	59.66$\pm$0.38
MSLR -WEB30K	Random Noise	57.88 $\pm$ 1.49	33.78 $\pm$ 6.31
	SimCL	64.21 $\pm$ 0.42	47.13 $\pm$ 0.78
	PiNDA $_{\mu=0}$	69.62$\pm$0.02	49.55 $\pm$ 0.11
	PiNDA $_{\mu \neq 0}$	69.60 $\pm$ 0.06	50.95$\pm$1.43
	CLAE	67.99 $\pm$ 0.78	51.54 $\pm$ 0.10
	PiNDA $_{\text{CLAE}}^{\mu=0}$	68.29 $\pm$ 0.53	52.18$\pm$0.90
	PiNDA $_{\text{CLAE}}^{\mu \neq 0}$	68.66$\pm$1.12	51.79 $\pm$ 0.77
Reuters -21578	Random Noise	38.30 $\pm$ 0.47	44.21 $\pm$ 0.14
	SimCL	38.00 $\pm$ 0.22	44.34 $\pm$ 0.40
	PiNDA $_{\mu=0}$	38.40 $\pm$ 0.06	44.37$\pm$0.27
	CLAE	38.95 $\pm$ 0.39	41.91 $\pm$ 0.10
	PiNDA $_{\text{CLAE}}^{\mu=0}$	39.18$\pm$0.26	42.07 $\pm$ 0.50

π -noise Setting In this paper, we attempt to learn Gaussian π -noise, *i.e.*,

$$p(\epsilon|\mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma}) \quad (18)$$

where $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$ represent the mean and variance respectively. To reduce the parameters to learn, we assume that $\boldsymbol{\Sigma}$ is a

diagonal matrix. In other words, $p(\epsilon|\mathbf{x})$ is an uncorrelated Gaussian distribution. On non-vision datasets, $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$ are both encoded by a 3-layer fully connected neural network, namely π -noise generator. Formally speaking, $(\boldsymbol{\mu}, \boldsymbol{\Sigma}) = \text{DNN}_{\psi}(\mathbf{x})$ where ψ is the learnable parameters. The hidden layers of the π -noise generator are all 1024. On vision datasets, they are represented by a ResNet18 to better extract the visual information.

After sampling standard Gaussian variable ϵ , the π -noise variable is obtained by the reparameterization trick, *i.e.*,

$$\epsilon = \boldsymbol{\mu} + \epsilon \odot \boldsymbol{\Sigma} \quad (19)$$

To study different kinds of noise, we also test the noise with a uniform distribution. where $\odot$ is Hadamard product operator.

For a data point $\mathbf{x}$, the noise of each dimension x_i obeys a independent uniform distribution, $p(\epsilon|x_i) = U(-u, u)$ where u is encoded by x_i . The experimental results are shown in Table 3 and Figure 4. The reparameterization trick is also used to retain the differentiability of noise sampling. Formally speaking, after sampling a noise ϵ from $U(0, 1)$, the π -noise is computed by $\epsilon = (2\epsilon - 1) \cdot u$.

To study whether the mean vector makes sense, we added the ablation experiments with fixing $\boldsymbol{\mu} = 0$. As shown by our experiments, the improvements of learnable $\boldsymbol{\mu}$ may be instable but it will result in much more parameters and calculations. So we suggest to fix $\boldsymbol{\mu}$ as 0 when the computational resources are limited. The codes are implemented under PyTorch and the experiments are conducted on a single GPU. The source codes are available at <https://github.com/hyzzhang98/PiNDA>.

4.3. Performance Analysis

4.3.1. RESULTS ON NON-VISION DATASETS

The classification accuracy evaluated by k NN and Softmax Regression on HAR and Reuters is reported in Table 2. From the table, it is not hard to find that PiNDA achieves better performance than baselines. For example, PiNDA achieves about 5% k NN improvement and 9% Softmax Regression improvement on Reuters than Random Noise. In particular, on Reuters, it results in severe instability to use the simple random noise as a main augmentation, which is quantitatively shown by variances. SimCL achieves relatively worse results, which may be caused by the normalization of added noise. The normalization also results in instability. However, we find that *the normalized π -noise is more stable in Figure 1*. The comparison of PiNDA and PiNDA with $\boldsymbol{\mu} = 0$ implies that the constraint of learnable mean vector (or namely bias) is frequently beneficial. It may suffer from the over-fitting problem when $\boldsymbol{\mu}$ is not constrained. The experiments with CLAE verify that PiNDA is

Table 3. Test accuracy (%) averaged over 5 runs on CIFAR-10 and CIFAR-100. Without specific statements, epochs of base models are set as 1000 and the batch size (denoted by m) is 256. Note that the performance gaps are mainly caused by the limitation of GPU, which limits the batch size in our experiments. But it still shows that PiNDA always promotes the base models.

Base Models	Setting	PiNDA Setting	Other Aug.	CIFAR-10		CIFAR-100	
				k NN	SR	k NN	SR
SimCLR	ResNet18, $m=256$	—	✗	28.89±0.46	36.92±0.36	5.93±0.37	9.58±0.24
		$\mathcal{N}(0, \Sigma)$	✗	32.48±0.48	37.57±0.98	6.46±0.33	10.13±0.17
		$\mathcal{N}(\mu, \Sigma)$	✗	31.83±0.58	36.72±0.93	5.50±0.38	9.86±0.09
	ResNet18, $m=256$	—	✓	86.05±0.21	91.98±0.71	37.56±0.80	70.43±0.27
		$\mathcal{N}(0, \Sigma)$	✓	86.60±0.39	92.25±0.07	37.62±0.64	70.64±0.44
		$\mathcal{N}(\mu, \Sigma)$	✓	87.75±0.60	90.50±0.47	38.86±0.11	68.96±0.11
	ResNet50, $m=64$	—	✓	66.38±0.50	50.91±0.53	30.60±1.06	59.33±1.78
		$\mathcal{N}(0, \Sigma)$	✓	70.00±1.30	52.20±0.34	30.27±0.50	62.00±1.93
	ResNet50, $m=256$	—	✓	74.43±0.52	84.81±0.91	43.69±1.00	57.67±2.20
		$\mathcal{N}(0, \Sigma)$	✓	75.00±0.12	86.48±0.97	45.09±0.99	59.97±1.33
		$U(-u, u)$	✓	74.52±0.29	85.49±0.54	44.16±0.49	59.37±3.31
	ResNet50, $m=512$	—	✓	75.65±0.46	85.52±1.85	44.59±0.11	58.70±2.60
		$\mathcal{N}(0, \Sigma)$	✓	76.41±0.30	88.50±0.62	46.30±0.44	60.58±1.72
MoCo	ResNet18	—	✓	71.31±0.50	72.22±0.21	41.50±0.38	35.13±0.31
		$\mathcal{N}(0, \Sigma)$	✓	71.71±0.19	72.56±0.28	41.05±0.30	35.70±0.17
		$\mathcal{N}(\mu, \Sigma)$	✓	73.50±0.41	74.16±0.30	43.18±0.19	40.76±0.13
BYOL	ResNet18, epoch=100	—	✓	71.08±0.78	78.55±0.30	34.05±1.23	49.04±0.65
		$\mathcal{N}(0, \Sigma)$	✓	71.18±0.27	78.65±0.29	34.54±1.08	49.42±0.69
		$\mathcal{N}(\mu, \Sigma)$	✓	71.16±0.13	78.43±0.25	37.75±0.34	48.97±0.38
	ResNet18, epoch=1000	—	✓	89.78±0.09	92.29±0.08	54.80±0.60	62.21±1.00
		$\mathcal{N}(0, \Sigma)$	✓	89.94±0.15	92.48±0.17	55.36±0.80	62.74±0.57
SimSiam	ResNet18, epoch=100	—	✓	70.51±0.47	74.11±0.43	37.91±0.21	54.02±0.46
		$\mathcal{N}(0, \Sigma)$	✓	70.52±0.33	74.40±0.33	38.07±0.14	54.40±0.09
		$\mathcal{N}(\mu, \Sigma)$	✓	70.86±0.46	74.30±0.51	42.71±0.16	52.30±0.12
	ResNet18, epoch=1000	—	✓	87.32±0.19	90.73±0.38	54.84±0.12	66.29±0.16
		$\mathcal{N}(0, \Sigma)$	✓	87.42±0.13	90.85±0.20	54.99±0.28	66.39±0.56
		$\mathcal{N}(\mu, \Sigma)$	✓	87.29±0.15	90.77±0.43	54.79±0.17	66.31±0.19
		$\mathcal{N}(\mu, \Sigma)$	✓				

Table 4. Test accuracy (%) averaged over 5 runs on Fashion-MNIST and STL-10. The batch size is set as 256. The subscripts and superscripts mean the base models and special settings.

Dataset	Method	k NN	SR
Fashion	SimCL	67.00±2.70	68.41±2.14
	Random	78.44±0.26	75.24±0.17
	PiNDA _{$\mu=0$}	79.35±0.93	79.09±1.16
	PiNDA _{$\mu \neq 0$}	79.08±0.96	78.95±0.87
	SimCL	27.23±0.72	34.52±1.29
	Random	33.17±0.59	38.24±0.21
	PiNDA _{$\mu=0$}	33.27±0.62	39.90±0.96
	PiNDA _{$\mu \neq 0$}	33.47±0.78	39.44±0.86
	BYOL	79.07±0.13	83.00±0.24
	PiNDA _{BYOL} _{$\mu=0$}	79.24±0.21	83.81±0.27
STL-10	PiNDA _{BYOL} _{$\mu \neq 0$}	79.47±0.56	83.52±0.06
	SimSiam	59.62±0.54	84.79±0.22
	PiNDA _{SimSiam} _{$\mu=0$}	77.18±0.21	84.80±0.15
	PiNDA _{SimSiam} _{$\mu \neq 0$}	77.57±0.17	84.31±0.12
	MoCo	72.13±0.65	76.84±0.32
	PiNDA _{MoCo} _{$\mu=0$}	72.19±0.25	77.09±0.25
	PiNDA _{MoCo} _{$\mu \neq 0$}	73.45±0.31	77.66±0.25
	SimCLR18	61.69±0.67	62.54±0.95
	PiNDA _{SimCLR18}	62.31±0.25	63.68±0.19
	PiNDA _{uniform} _{SimCLR18}	61.30±0.29	63.28±1.17
	SimCLR18 _{$m=512$}	64.92±1.14	65.98±1.78
	PiNDA _{SimCLR18} _{$m=512$}	65.55±0.94	67.28±0.89
	SimCLR50	64.43±0.25	75.03±0.19
	PiNDA _{SimCLR50}	65.47±0.36	75.25±0.26

Table 5. Test accuracy (%) on ImageNet. The batch size is set as 256. Due to the limitation of computational resources, the base models are only trained within 100 epochs.

Backbone	Base	PiNDA Setting	k NN	SR
ResNet18	BYOL	—	43.04	56.37
		$\mathcal{N}(0, \Sigma)$	43.25	56.57
	SimSiam	—	42.96	56.55
		$\mathcal{N}(0, \Sigma)$	43.11	56.78
	MoCo	—	50.83	64.25
		$\mathcal{N}(0, \Sigma)$	51.10	64.58
	SimCLR	—	52.05	64.18
Transformer	DINO	$\mathcal{N}(0, \Sigma)$	55.48	66.86
		—	55.44	60.65
Transformer	DINO	$\mathcal{N}(0, \Sigma)$	55.50	61.22
		—	55.44	60.65

compatible with the existing contrastive methods and *it can be an extra module for any popular contrastive models* to further promote the performance and stabilize the model.

4.3.2. RESULTS ON VISION DATASETS

The classification accuracy evaluated by k NN and Softmax Regression on vision datasets is summarized in Tables 3, 4, and 5. Similar to results on non-vision datasets, PiNDA outperforms other baselines. For example, PiNDA increases Random Noise by almost 4 percent under k NN evaluation on CIFAR-10 and 4 percent under Softmax Regression

on Fashion-MNIST. PiNDA improves the SimCLR with ResNet50 on CIFAR-100 more than 2 percent under all settings. It also increases MoCo by more than 5 percent. More importantly, PiNDA with SimSiam causes almost 20 percent improvement on STL-10 under the k NN evaluation. Note that the reason why the baselines cannot achieve the similar results reported by their original papers is the limited batch size, which is greatly **limited by computational resources**. The batch size of SimCLR is only set as 64/256/512 while SimCLR is sensitive to batch size. SimCLR with different backbones are also shown in Table 3. Overall, from our extensive experiments, we found that *PiNDA is stable technique to augment the existing contrastive learning models under diverse settings*.

4.3.3. VISUALIZATION

The learned π -noise is visualized in Figures 1 and 4. As discussed above, we mainly report PiNDA with $\mu = 0$. In Figure 1, the noise is first normalized and then fed to contrastive models. In Figure 4, the noise is not normalized. Meanwhile in Figure 4, we visualize the different settings of π -noise, including Gaussian noise and uniform noise ($\mathcal{N}(0, \Sigma)$, $\mathcal{N}(\mu, \Sigma)$, and $U(-u, u)$). Due to the limitation of pages, more visualizations with different backbones and more discussions can be found in Appendix C.3.

4.3.4. DIFFERENT NOISE GENERATORS

To study the impact of different implementations of the noise generator, the experiments of SimCLR on two high-resolution vision datasets, STL-10 and ImageNet, are conducted and the experimental results are shown in Table 6. We employ ResNet18, ResNet50, and U-Net (Ronneberger et al., 2015) as the generator.

Table 6. Comparison with different implementation of noise generator on STL-10 and ImageNet.

Dataset	Generator	k NN	SR	Time	FLOPs	Space
STL-10 (ResNet18)	—	61.69	62.54	21s	2.57e12	48G
	ResNet18	62.31	63.68	+21s	+2.60e12	+20G
	ResNet50	54.72	61.24	+45s	+6.08e12	+133G
	U-Net	63.57	63.95	+20s	+4.73e12	+22G
STL-10 (ResNet50)	—	52.56	63.72	46s	6.05e12	151G
	ResNet18	51.84	64.31	+18s	+2.60e12	+25G
	ResNet50	54.59	65.85	+25s	+6.00e12	+113G
	U-Net	55.92	63.48	+19s	+4.80e12	+33G
ImageNet (ResNet18)	—	52.05	64.18	3min	2.57e12	48G
	ResNet18	55.48	66.86	+4min	+2.60e12	+20G
	ResNet50	45.18	58.53	+10min	+6.08e12	+133G
	U-Net	43.01	58.28	+4min	+4.73e12	+32G
ImageNet (ResNet50)	—	52.03	65.85	9min	6.05e12	151G
	ResNet18	53.18	66.34	+5min	+2.60e12	+25G
	ResNet50	57.41	67.70	+10min	+6.00e12	+113G
	U-Net	56.04	67.40	+4min	+4.80e12	+33G

4.3.5. EXPERIMENTS OF BATCH SIZE

As shown in SimCLR (Chen et al., 2020), larger batch size usually means better performance. However, due to the

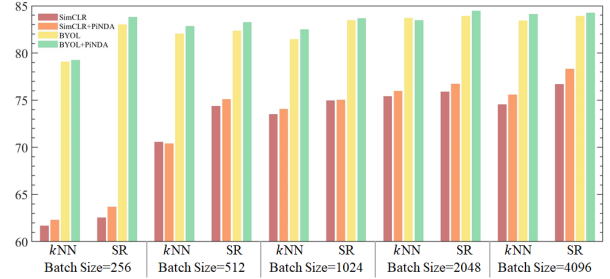


Figure 5. Test accuracy on STL10 with different batch sizes. Due to the computational limitation, the models are only trained within 100 epochs and the images are resized to $3 \times 32 \times 32$.

limitation of computational resources, we cannot reproduce many contrastive models with the large batch size as reported in the original papers. We try our best to conduct experiments with different batch sizes in Tables 2–5. Besides, experiments on STL-10 with several batch sizes are conducted to demonstrate the performance of PiNDA with different batch sizes. The results are shown in Figure 5. Note that due to the limitation of space, the images are resized to $3 \times 32 \times 32$ during the training.

5. Conclusion

In this paper, we design an auxiliary Gaussian distribution related to the contrastive loss to define the task entropy, the core of π -noise framework, of contrastive learning. With the definition of contrastive task entropy, we prove that the predefined data augmentations in the standard contrastive framework are equivalent to the point estimation of π -noise. Guided by the theoretical analysis, we propose to learn the π -noise instead of estimation. The learning of π -noise has two significant merits: (1) Since random noise is a common augmentation in existing contrastive learning, the proposed π -noise generator is completely compatible with the existing contrastive learning. Experiments show the existence and effectiveness of π -noise. (2) The generation of π -noise does not depend on the type of data and can be applied to any kind of data. The proposed method therefore helps extend contrastive learning to other fields. More surprisingly, we find that the developed π -noise generator successfully learns augmentations on vision datasets, which is similar to style transfer results. It indicates the potential of PiNDA and it deserves further investigation.

One may concern the additional burden of PiNDA. As shown in Table 11, PiNDA obtains better results than the original contrastive models under the similar computational expenses. In addition, as indicated by Table 6, the generator implemented the same architecture as the base model works well in most cases. In the future studies, a promising scheme is to share the parameters between the generator and base model, which can significantly reduce the extra expenses.

Impact Statement

This paper presents work whose goal is to advance the field of machine learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

References

- Bishop, C. M. *Pattern recognition and machine learning, 5th Edition*. Information science and statistics. Springer, 2007. ISBN 9780387310732.
- Cao, J., Nai, R., Yang, Q., Huang, J., and Gao, Y. An empirical study on disentanglement of negative-free contrastive learning. In *NeurIPS*, 2022.
- Caron, M., Touvron, H., Misra, I., Jegou, H., Mairal, J., Bojanowski, P., and Joulin, A. Emerging properties in self-supervised vision transformers. In *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 9630–9640, 2021.
- Chen, T., Kornblith, S., Norouzi, M., and Hinton, G. E. A simple framework for contrastive learning of visual representations. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119, pp. 1597–1607, 2020.
- Chen, X. and He, K. Exploring simple siamese representation learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 15750–15758, 2021.
- Cheung, T. and Yeung, D. MODALS: modality-agnostic automated data augmentation in the latent space. In *9th International Conference on Learning Representations*, 2021.
- Coates, A., Ng, A. Y., and Lee, H. An analysis of single-layer networks in unsupervised feature learning. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, volume 15, pp. 215–223, 2011.
- Grill, J., Strub, F., Altché, F., Tallec, C., Richemond, P. H., Buchatskaya, E., Doersch, C., Pires, B. Á., Guo, Z., Azar, M. G., Piot, B., Kavukcuoglu, K., Munos, R., and Valko, M. Bootstrap your own latent - A new approach to self-supervised learning. In *Advances in Neural Information Processing Systems*, 2020.
- Gutmann, M. and Hyvärinen, A. Noise-contrastive estimation: A new estimation principle for unnormalized statistical models. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, volume 9, pp. 297–304, 2010.
- Hassani, K. and Khasahmadi, A. H. Contrastive multi-view representation learning on graphs. In *International Conference on Machine Learning*, pp. 4116–4126, 2020.
- He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 770–778, 2016.
- He, K., Fan, H., Wu, Y., Xie, S., and Girshick, R. B. Momentum contrast for unsupervised visual representation learning. In *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 9726–9735, 2020.
- Ho, C. and Vasconcelos, N. Contrastive learning with adversarial examples. In *Advances in Neural Information Processing Systems*, 2020.
- Hu, Z., Xu, Q., Duan, Y., Tai, Y., and Li, H. Sota: Self-adaptive optimal transport for zero-shot classification with multiple foundation models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2026.
- Huang, S., Zhang, H., and Li, X. Enhance vision-language alignment with noise. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pp. 17449–17457, 2025.
- Jiao, Z., Zhang, H., and Li, X. Deep graph multi-view representation learning with self-augmented view fusion. *IEEE Transactions on Neural Networks and Learning Systems*, 2025.
- Kingma, D. P. and Ba, J. Adam: A method for stochastic optimization. In *3rd International Conference on Learning Representations*, 2015.
- Kingma, D. P. and Welling, M. Auto-encoding variational bayes. In *2nd International Conference on Learning Representations*, 2014.
- Kirillov, A., Mintun, E., Ravi, N., Mao, H., Rolland, C., Gustafson, L., Xiao, T., Whitehead, S., Berg, A. C., Lo, W.-Y., et al. Segment anything. In *Proceedings of the IEEE/CVF international conference on computer vision*, pp. 4015–4026, 2023.
- Lewis, D. D., Yang, Y., Rose, T. G., and Li, F. RCV1: A new benchmark collection for text categorization research. *J. Mach. Learn. Res.*, 5:361–397, 2004.
- Li, X. Positive-incentive noise. *IEEE Transactions on Neural Networks and Learning Systems*, pp. 1–7, 2022. doi: 10.1109/TNNLS.2022.3224577.
- Pan, E. and Kang, Z. Multi-view contrastive graph clustering. In *Advances in Neural Information Processing Systems*, pp. 2148–2159, 2021.

- PASCAL. Pascal large scale learning challenge, 2008. URL <https://www.csie.ntu.edu.tw/~cjlin/libsvmtools/datasets/binary.html>.
- Qin, T. and Liu, T.-Y. Introducing letor 4.0 datasets. *arXiv preprint arXiv:1306.2597*, 2013.
- Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., and Sutskever, I. Learning transferable visual models from natural language supervision. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139, pp. 8748–8763, 2021.
- Reyes-Ortiz, J., Anguita, D., Ghio, A., Oneto, L., and Parra, X. Human Activity Recognition Using Smartphones. UCI Machine Learning Repository, 2012. DOI: <https://doi.org/10.24432/C54S4K>.
- Robinson, J. D., Chuang, C., Sra, S., and Jegelka, S. Contrastive learning with hard negative samples. In *International Conference on Learning Representations*, 2021.
- Ronneberger, O., Fischer, P., and Brox, T. U-net: Convolutional networks for biomedical image segmentation. In *Medical image computing and computer-assisted intervention (MICCAI 2015)*, pp. 234–241, 2015.
- Russakovsky, O., Deng, J., Su, H., Krause, J., Satheesh, S., Ma, S., Huang, Z., Karpathy, A., Khosla, A., Bernstein, M., et al. Imagenet large scale visual recognition challenge. *International journal of computer vision*, 115: 211–252, 2015.
- Torralba, A., Fergus, R., and Freeman, W. T. 80 million tiny images: A large data set for nonparametric object and scene recognition. *IEEE Trans. Pattern Anal. Mach. Intell.*, 30(11):1958–1970, 2008.
- van den Oord, A., Li, Y., and Vinyals, O. Representation learning with contrastive predictive coding. *CoRR*, abs/1807.03748, 2018.
- Verma, V., Luong, T., Kawaguchi, K., Pham, H., and Le, Q. V. Towards domain-agnostic contrastive learning. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139, pp. 10530–10541, 2021.
- Wang, J., Zhang, H., and Yuan, Y. Adv-cpg: A customized portrait generation framework with facial adversarial attacks. *arXiv preprint arXiv:2503.08269*, 2025.
- Wu, L., Liu, Z., Xia, J., Zang, Z., Li, S., and Li, S. Z. Generalized clustering and multi-manifold learning with geometric structure preservation. In *IEEE/CVF Winter Conference on Applications of Computer Vision*, pp. 1668–1676, 2022.
- Wu, Z., Xiong, Y., Yu, S. X., and Lin, D. Unsupervised feature learning via non-parametric instance discrimination. In *IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2018*, pp. 3733–3742, 2018.
- Xiao, H., Rasul, K., and Vollgraf, R. Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms. *CoRR*, abs/1708.07747, 2017.
- Yang, H., Ding, X., Wang, J., and Li, J. Simcl: Simple contrastive learning for image classification. In *5th International Conference on Big Data Technologies*, pp. 273–278, 2022.
- Zhang, H., Huang, S., Guo, Y., and Li, X. Variational positive-incentive noise: How noise benefits models. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, abs/2306.07651, 2023. doi: 10.48550/arXiv.2306.07651.
- Zhu, Y., Xu, Y., Yu, F., Liu, Q., Wu, S., and Wang, L. Graph contrastive learning with adaptive augmentation. In *WWW ’21: The Web Conference 2021*, pp. 2069–2080, 2021.

A. Derivation of Eq. (13)

In this part, we show how to get Eq. (13). Substitute the PDF of $p(\alpha|\mathbf{x}, \varepsilon_0)$ into $\mathcal{L}$, we have

$$\begin{aligned}\mathcal{L} &= \frac{1}{n} \sum_{\mathbf{x}} \int p(\alpha|\mathbf{x}, \varepsilon_0) \cdot (\log C + \frac{1}{2} \log \gamma_{\theta^*}(\mathbf{x}, \varepsilon_0) - \frac{\alpha^2}{2} \cdot \gamma_{\theta^*}(\mathbf{x}, \varepsilon_0)) d\alpha \\ &= \frac{1}{n} \sum_{\mathbf{x}} \log C + \frac{1}{2} \log \gamma_{\theta^*}(\mathbf{x}, \varepsilon_0) - \frac{\gamma_{\theta^*}(\mathbf{x}, \varepsilon_0)}{2} \int \alpha^2 p(\alpha|\mathbf{x}, \varepsilon_0) d\alpha \\ &= \frac{1}{n} \sum_{\mathbf{x}} \log C + \frac{1}{2} \log \gamma_{\theta^*}(\mathbf{x}, \varepsilon_0) - \frac{1}{2}.\end{aligned}\tag{20}$$

The final step is derived from $\int \alpha^2 p(\alpha|\mathbf{x}, \varepsilon_0) d\alpha = \mathbb{E}(\alpha - \mathbb{E}\alpha)^2 = \text{variance}(\alpha) = \gamma_{\theta^*}(\mathbf{x}, \varepsilon_0)^{-1}$.

B. More Details of Experimental Settings

B.1. Datasets

In our experiments, there are 4 non-vision datasets used for validating the effectiveness of PiNDA on non-vision data: HAR (Reyes-Ortiz et al., 2012) is collected by sensors of smartphones; Reuters (Lewis et al., 2004) is a subset of the original dataset and the preprocessing is the same with (Wu et al., 2022); epsilon (PASCAL, 2008) is a dataset used for PASCAL Challenge 2008; MSLR-WEB30K (Qin & Liu, 2013), consists of feature vectors extracted from query-url pairs along with relevance judgment labels, is released by Microsoft. Besides, 5 vision datasets, including Fashion-MNIST (Xiao et al., 2017), CIFAR-10 (Torralba et al., 2008), CIFAR-100 (Torralba et al., 2008), STL-10 (Coates et al., 2011), and ImageNet (Russakovsky et al., 2015), are also used for evaluation. On the one hand, *the experiments on these vision datasets provide the better visualization of π -noise augmentation*. On the other hand, *the experiments also validates the effectiveness and compatibility of PiNDA to the existing contrastive models*. The details are summarized in Table 1. Without additional clarifications, the images of STL-10 are not resized and the images of ImageNet are resized to $3 \times 96 \times 96$.

B.2. Baseline

SimCL (Yang et al., 2022) proposes to add the normalized noise to the learned representations to obtain augmentations and feed the noisy augmentations into the projection head to compute the contrastive loss. Therefore, it is an important baseline. To better *verify whether the idea of learning π -noise works*, a compared method that utilizes the untrained random Gaussian noise is also added as a competitor method, denoted by *Random Noise*. As the value range of original features of various datasets differs a lot, we first rescale the input feature and then add the standard Gaussian noise to the features. *PiNDA* denotes the contrastive model with only a π -noise augmentation. Since *CLAE* (Ho & Vasconcelos, 2020) uses adversarial examples as an augmentation and is not limited to non-vision datasets, it is another important baseline. *SimCLR* (Chen et al., 2020), *BYOL* (Grill et al., 2020), *SimSiam* (Chen & He, 2021), and *MoCo* (He et al., 2020) also serve as important baselines on vision data.

We modify the baselines by using π -noise as an additional augmentation for contrast, which are marked by subscripts. They are also conducted to verify whether PiNDA is compatible with other contrastive models. For all baselines, the backbone used for extracting contrastive representations is the same by default. The backbone networks on non-vision and vision datasets are 3-layer DNN and ResNet18, respectively.

Following SimCLR, we also use the projection head, which consists of a 256-dimension fully connected layer with ReLU activation and a 128-dimension linear layer, to minimize the contrastive loss and use the input of the projection head as the learned representation.

B.3. Evaluation Settings

The extracted contrastive embeddings are testified by two simple machine learning models, softmax regression (a neural network composed of a linear layer and a softmax layer) and k -nearest neighbors (k NN). For the softmax regression, the epoch is set as 50 and the batch size is 256. and the learning rate is set as 0.001. For k NN, k is set as 5. All experiments share the same settings. The split of the training set and test set is following the standard split of the datasets.

B.4. Contrastive Loss

On non-vision datasets, the implementation details of the contrastive loss are the same as SimCLR. On vision datasets, the contrastive loss consists of the original loss according to the base model used and an additional penalty term, which is the multiplicative inverse of the average norm of the π -noises, ensuring that the norm of the π -noise will not be too small. The loss is optimized by the Adam algorithm (Kingma & Ba, 2015) with a learning rate of 10^{-3} . The temperature coefficient τ in the contrastive loss is set to 0.1 on all datasets.

C. More Experimental Results

In this section, we show more experimental results, visualizations, and experimental analysis. In Section C.2, the experiments with SAM (Kirillov et al., 2023) are reported, which serves as another baseline. In Section C.5, the additional costs of PiNDA with different settings are reported.

C.1. More Analysis and Conclusions on Vision Datasets

From the experiments on vision datasets, it is clearly shown that $\mu = 0$ may be a cost-efficient setting in practice when π -noise is assume as Gaussian noise. It seems that the learnable μ will not bring further improvements in all cases but results in much more learnable parameters and computations compared with the setting $\mu = 0$. Therefore, we recommend to set $\mu = 0$ when the computational resources are limited.

C.2. Comparison with SAM

From the visualization shown by Figures 1, 4, 6, and 7, we found that the noise generator can detect the irrelevant background and main objects. A direct idea is to compare PiNDA with Segment Anything Model (SAM) (Kirillov et al., 2023).

Table 7. Test accuracy (%) averaged over 5 runs on CIFAR-10, CIFAR-100, and STL-10. Epochs of base models are set as 1000.

Backbone	Other Augmentations	PiNDA Setting	CIFAR-10		CIFAR-100		STL-10	
			kNN	SR	kNN	SR	kNN	SR
ResNet18, $m=256$	Default	—	86.05 $\pm$ 0.21	91.98 $\pm$ 0.71	37.56 $\pm$ 0.80	70.43 $\pm$ 0.27	61.69 $\pm$ 0.67	62.54 $\pm$ 0.95
	SAM	—	66.61 $\pm$ 0.43	90.31 $\pm$ 0.39	35.06 $\pm$ 0.64	68.90 $\pm$ 0.26	52.68 $\pm$ 1.53	62.78 $\pm$ 1.15
	PiNDA	$\mathcal{N}(0, \Sigma)$	86.60$\pm$0.39	92.25$\pm$0.07	37.62$\pm$0.64	70.64$\pm$0.44	62.31$\pm$0.25	63.68$\pm$0.19
ResNet50, $m=256$	Default	—	74.43 $\pm$ 0.52	84.81 $\pm$ 0.91	43.69 $\pm$ 1.00	57.67 $\pm$ 2.20	64.43 $\pm$ 0.25	75.03 $\pm$ 0.19
	SAM	—	68.35 $\pm$ 0.56	84.36 $\pm$ 2.09	37.81 $\pm$ 0.80	57.56 $\pm$ 2.13	72.83$\pm$3.58	75.04 $\pm$ 2.70
	PiNDA	$\mathcal{N}(0, \Sigma)$	75.00$\pm$0.12	86.48$\pm$0.97	45.09$\pm$0.99	59.97$\pm$1.33	65.47 $\pm$ 0.36	75.25$\pm$0.26

Table 8. Test accuracy (%) on ImageNet. The batch size is set as 256. The base models are only trained within 100 epochs.

Backbone	Base	Aug	Setting	kNN	SR
ResNet18	SimCLR	Default	—	52.05	64.18
		SAM	—	55.41	66.95
		PiNDA	$\mathcal{N}(0, \Sigma)$	55.48	66.86

We use SAM to segment the background and then add noise to the background with the highest probability. All processing with SAM is conducted before the training as a pre-processing step. The experiments are summarized in Table 7 and Table 8. From the two additional tables, we found that in most cases, PiNDA is more effective than SAM. It should be emphasized that **the pre-processing of SAM on ImageNet consumes more than 19 hours (about 6 times of training time)**, though it outperforms PiNDA on ImageNet. In contrast, the additional time required by PiNDA is only the half of training time.

C.3. More Visualization and Discussions

In Figure 6, we show the unnormalized Gaussian π -noise with SimCLR on STL10. In Figure 7, we visualize the noise learned by a different contrastive model, BYOL.

Surprisingly, after sampling a π -noise and adding it to the original images, we obtain some remarkable augmentations. As shown in the figures, the learned π -noise indicates great effect toward the background of the image while mere effect toward the target, which leads to the bigger chance for the base model to be relevant to the target itself when extracting

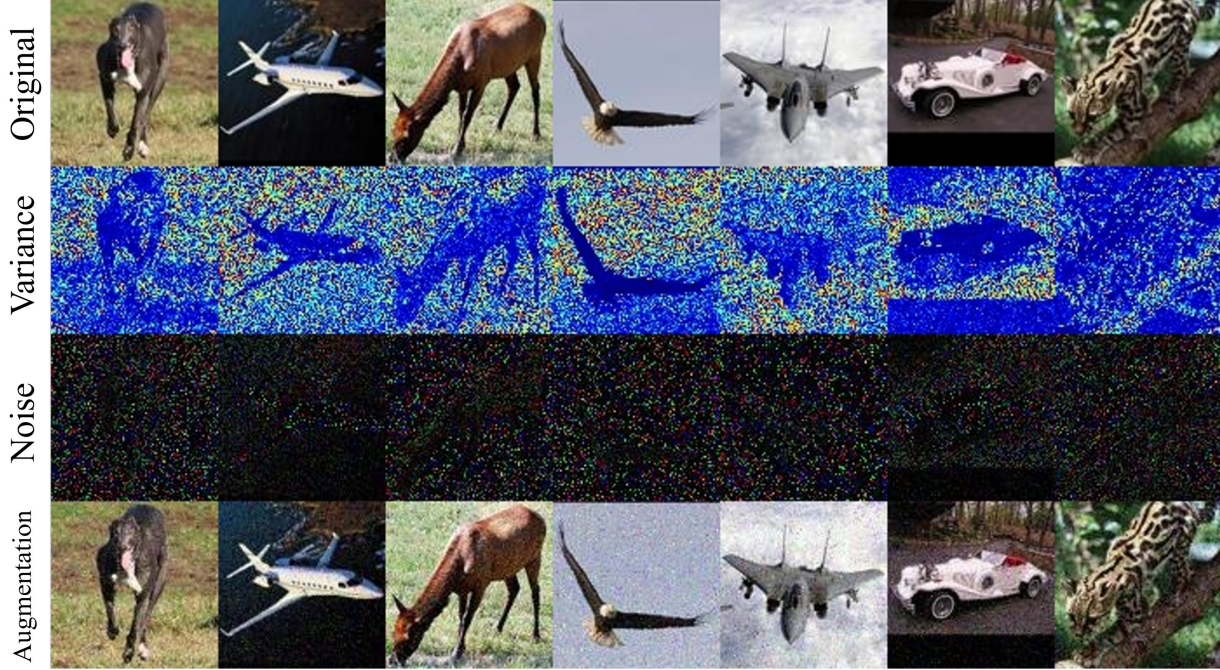


Figure 6. More visualization of the π -noise by PiNDA trained with SimCLR on STL-10. We fix μ as 0 and only learn the variance of Gaussian π -noise, which is visualized in the second row. The third row is a sampling result from the learned π -noise distribution. Compared with Figure 1, the noise is not normalized.

features. For example, in Figure 1, the rightmost image (horse) is augmented by changing the sky (bluer) and ground (from grassland to desert), which is extremely similar to the style transfer result. Nevertheless, with only the original image inputted into the networks, the π -noise generator extracts the effective visual information and produce strong augmentations, which significantly validates the idea. The visualization also indicates the great potential and underlying applications of the proposed π -noise generators. In Figure 6, we found that the noise generator can still effectively learn the outline of objects.

More surprisingly, in Figure 4, we found that the learnable μ contains no outline information and only Σ contains the useful information. It partially explains why learnable μ will not always improve the performance, which is shown in Tables 3 and 4. We also found that the uniform π -noise generator can also learn the outline but the visualization is not good as Gaussian π -noise.

From Figure 7, we find that PiNDA with other contrastive models (even without negative samples) can also learn the outline of objects in an unsupervised way.

Furthermore, the learned π -noise demonstrates the ability to differ the background from target that has similar color. For example, in the rightmost image (cat) of Figure 6, the variance shows the trend of the noise to influence the branch much more than to influence the cat. The same occasion also occurs in the rightmost two images of Figure 7. In the images, although the main body of the monkey and the car have similar color to the surrounding, the visualization of variance indicates that PiNDA successfully differs them from the background.

C.4. More Analysis about Different Noise Generators (Table 6)

To study the impact of different implementations of the noise generator, the experiments of SimCLR on two high-resolution vision datasets, STL-10 and ImageNet, are conducted and the experimental results are shown in Table 6. We employ ResNet18, ResNet50, and U-Net (Ronneberger et al., 2015) as the generator.

From Table 6, we can conclude that ResNet-18 is the most appropriate models from the aspects of both performance and costs. Although ResNet-50 outperforms ResNet-18 in some cases, it requires too much additional overhead but the improvement is relatively limited. U-Net is harder to train on ImageNet and results in more computational overhead compared with ResNet-18.

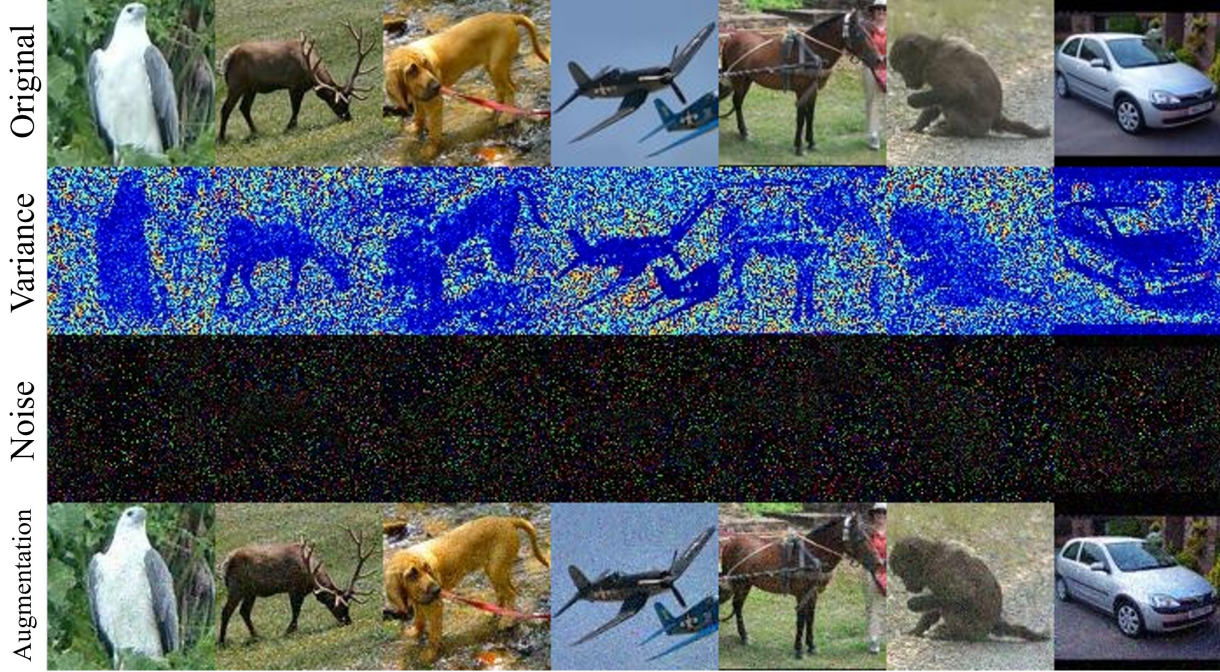


Figure 7. Visualization of the π -noise by PiNDA trained with BYOL on STL-10. Similarly, we fix μ as 0 and only learn the variance of Gaussian π -noise. We find that PiNDA can effectively learn the outline with different contrastive framework.

Table 9. Additional computational costs of SimCLR. Time: Training time per epoch. Memory: GPU memory. Batch size: 256.

Generator	CIFAR-10			CIFAR-100			STL-10			ImageNet		
	Time	FLOPs	Memory	Time	FLOPs	Memory	Time	FLOPs	Memory	Time	FLOPs	Memory
Baseline	39s	1.32e9	15.85G	39s	1.32e9	15.85G	21s	2.57e12	48.20G	3min	2.57e12	48.20G
DNN3	+5s	+1.26e7	+0.23G	+5s	+1.26e7	+0.23G	+11s	+5.64e9	+11.47G	+2min	+5.64e9	+11.47G
ResNet18	+11s	+5.65e8	+2.26G	+11s	+5.65e8	+2.26G	+21s	+2.60e12	+20.15G	+4min	+2.60e12	+20.15G
ResNet50	+36s	+1.32e9	+12.55G	+36s	+1.32e9	+12.55G	+45s	+6.08e12	+133.03G	+10min	+6.08e12	+133.03G

C.5. Additional Cost

Since PiNDA works as an additional module of contrastive models, the extra computational costs of training noise generator are important in practice. We report the additional costs to SimCLR with different implementations of noise generators on 4 vision datasets in Table 9. The additional costs to a DNN3 on 4 non-vision datasets are also reported in Table 10. The costs are evaluated by 3 different metrics: training time, floating point of operations (FLOPs), and required GPU memory. Note that the noise generator is implemented by ResNet18 in all other experiments though we test the computational costs of ResNet-50 as well. If we use ResNet-50 as the generator, the out-of-memory (OOM) exception will occur in many cases.

Table 10. Additional computational costs on non-vision datasets.

Dataset	Generator	Time	FLOPs	Memory
HAR	Baseline	0.23s	4.99e8	900M
	PiNDA	+0.08s	+1.25e9	+84M
Reuters	Baseline	0.43s	8.76e8	1044M
	PiNDA	+0.05s	+2.38e9	+146M
Epsilon	Baseline	17.23s	1.73e9	7.04G
	PiNDA	+0.90s	+2.38e9	+1.12G
MSLR -WEB30K	Baseline	136.56s	3.88e8	6.72G
	PiNDA	+55.45s	+9.12e8	+0.32G

C.5.1. PERFORMANCE COMPARISONS WITH SIMILAR BURDEN

We conduct further experiments to show the effectiveness of PiNDA with low computational cost. The results are shown in Table 11. It is easy to find that PiNDA can achieve better results compared with the original models with larger batch size,

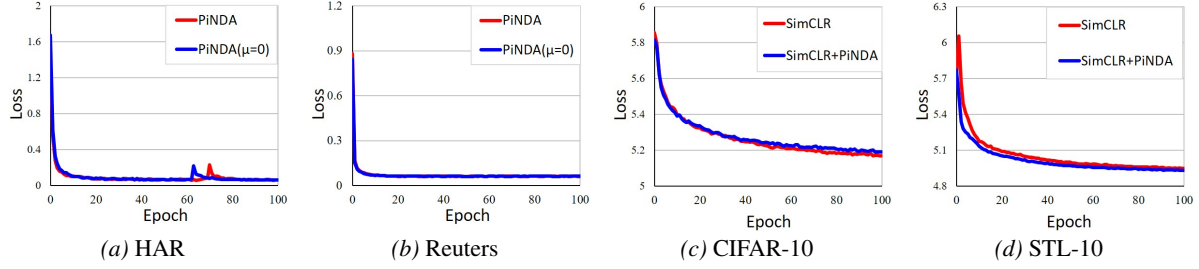


Figure 8. Loss curves on two non-vision datasets and two vision datasets. From the figures, we find that the loss of PiNDA decreases rapidly and converges fast.

Table 11. Comparison of accuracy of SimCLR and SimCLR with PiNDA under similar memory burden condition.

	CIFAR-10				CIFAR-100			
	Batch Size	Memory	k NN	SR	Batch Size	Memory	k NN	SR
SimCLR	256	15.85G	74.43 $\pm$ 0.52	84.81 $\pm$ 0.91	256	15.70G	43.59 $\pm$ 1.00	57.67 $\pm$ 2.20
	300	19.06G	72.44 $\pm$ 0.23	85.76 $\pm$ 0.55	300	19.07G	43.83 $\pm$ 0.83	58.82 $\pm$ 1.60
PiNDA _{SimCLR}	256	18.11G	75.00$\pm$0.12	86.48$\pm$0.97	256	18.10G	45.09$\pm$0.99	59.97$\pm$1.33

when they have the same cost. It indicates that **the improvement brought by PiNDA is more effective and direct than the benefits from larger batch.**

C.6. Convergence

To study the convergence of PiNDA, the loss curves on HAR, Reuters, CIFAR-10, and STL-10 are shown in Figure 8. From the figure, it is easy to find that the loss of PiNDA decreases rapidly. On STL-10, PiNDA helps the base SimCLR converge more rapidly. Although the loss is not a fair metric to reflect the performance, it empirically verifies the convergence of PiNDA.